

Nome: Felipe Meiga Matrícula: 2011460

1) Conversão de múltiplos loops `while` para um único `while`

Na programação, há duas principais estruturas de loop, o `for` e o `while`. O `for` pode ser interpretado como uma versão mais sintaticamente sofisticada do `while`. Sendo assim, tudo que é possível fazer com um `for` é possível fazer com um `while`.

Para transformar um código com múltiplos loops para utilizar apenas a estrutura `while`, podemos consolidar todos os loops em um único `while`, onde a condição de repetição é a união (disjunção) de todas as condições dos `while` internos. Dentro do `while` único, podemos usar estruturas condicionais, como `if`, para determinar qual bloco de código deve ser executado em cada iteração. Assim, o código resultante mantém a lógica dos `while` originais.

2) Teorema da Forma Normal de Kleene

O teorema da forma normal de Kleene afirma que qualquer função computável pode ser reduzida a um único `while`. No contexto do cálculo lambda, isso significa que podemos expressar qualquer função com apenas uma instância do operador Ω .

Uma maneira de demonstrar esse teorema é usando o conceito de uma máquina de Turing: a execução de uma máquina de Turing pode ser vista como um único `while` que continua para sempre ou até a máquina concluir. Como o cálculo lambda permite a construção de um interpretador para uma máquina de Turing, conseguimos representar outros sistemas computacionais que também são completos em relação a Turing, reforçando essa equivalência.

3) Definição de Listas de Aridade Arbitrária

Para criar listas de aridade arbitrária, utilizaremos as definições de booleanos e pares ordenados (juntamente com seus operadores) apresentadas em aula:

Booleanos

$$\text{TRUE} := \lambda x. \lambda y. x$$
$$\text{FALSE} := \lambda x. \lambda y. y$$

$$\text{ITE} := \lambda e. \lambda t. \lambda f. e \, t \, f$$

Pares Ordenados

$$\text{PAIR} := \lambda x. \lambda y. \lambda z. z \, x \, y$$

$$\text{FST} := \lambda p. p \, \text{TRUE}$$

$$\text{SND} := \lambda p. p \, \text{FALSE}$$

Definição de Listas Encadeadas

Com essas definições, podemos criar uma lista de aridade arbitrária utilizando a ideia de uma lista encadeada. Usando pares ordenados, representamos o primeiro elemento da lista como o primeiro valor do par e o restante da lista como o segundo valor. Esse segundo valor pode ser outro par (representando a continuação da lista) ou um marcador especial que indica o fim da lista. O marcador do fim também será considerado uma lista válida, representando a lista vazia.

Representação da Lista Vazia

Para representar a lista vazia, definimos o símbolo (função) `NIL`, e para verificar se uma lista está vazia, usamos a função `NIL?`:

$$\text{NIL} := \lambda x. \text{TRUE}$$

$$\text{NIL?} := \lambda p. p \, (\lambda x. \lambda y. \text{FALSE})$$

A lógica dessa definição é que, se a lista for vazia (representada por `TRUE`), a função `NIL?` retornará `TRUE`. Caso contrário, ela retornará `FALSE`.

Exemplo de Uso

Ao verificar uma lista vazia:

$$\text{NIL? NIL}$$

Substituímos `NIL`:

$$\text{NIL?} (\lambda x. \text{TRUE})$$

Aplicamos a função `NIL?`:

$$(\lambda p. p \, (\lambda x. \lambda y. \text{FALSE})) (\lambda x. \text{TRUE})$$

Substituímos o parâmetro p :

$$(\lambda x. \text{TRUE}) (\lambda x. \lambda y. \text{FALSE})$$

Aplicamos a função:

$$\text{TRUE}$$

Ao passar uma lista não vazia (um par contendo algum elemento), a função NIL? retornará FALSE.

Com essa definição, é possível implementar um algoritmo para percorrer a lista. Assim, qualquer algoritmo que utilize listas pode ser desenvolvido com base nessa representação de listas encadeadas.

4) Definições de funções no Cálculo Lambda

$$\text{TRUE} = \lambda x. \lambda y. x$$

$$\text{FALSE} = \lambda x. \lambda y. y$$

$$\text{IF} = \lambda p. \lambda a. \lambda b. p a b$$

$$\text{ISZERO} = \lambda n. n (\lambda x. \text{FALSE}) \text{TRUE}$$

$$\text{ADD} = \lambda m. \lambda n. \lambda f. \lambda x. m f (n f x)$$

$$\text{PRED} = \lambda n. \lambda f. \lambda x. n (\lambda g. \lambda h. h (g f)) (\lambda u. x) (\lambda u. u)$$

$$\text{SUCC} = \lambda n. \lambda f. \lambda x. f (n f x)$$

$$\text{OR} = \lambda p. \lambda q. p \text{TRUE } q$$

$$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

$$\text{SUM} = Y (\lambda s. \lambda n. \text{IF} (\text{ISZERO } n) \text{ZERO} (\text{ADD } n (s (\text{PRED } n))))$$

Descrição:

- Caso base: quando $n = 0$, a soma é 0.
- Caso recursivo: quando $n > 0$, temos $\text{SUM}(n) = n + \text{SUM}(n - 1)$.

O combinador Y permite que a função que recebe s chame a si mesma. O bloco ISZERO verifica se $n = 0$, e ADD soma n ao resultado da chamada recursiva.

$$\begin{aligned} \text{FIB} = Y (\lambda f. \lambda n. \text{IF} (\text{ISZERO } n) \text{ZERO} \\ (\text{IF} (\text{ISZERO} (\text{PRED } n)) \text{SUCC}(\text{ZERO}) \\ (\text{ADD} (f (\text{PRED } n)) (f (\text{PRED} (\text{PRED } n)))))) \end{aligned}$$

Descrição:

- Caso base 1: quando $n = 0$, o resultado é 0.
- Caso base 2: quando $n = 1$, o resultado é 1.
- Caso recursivo: quando $n \geq 2$, temos $FIB(n) = FIB(n-1) + FIB(n-2)$.

5) Definição da Função de Ackermann no Cálculo Lambda

A função de Ackermann é um exemplo clássico de uma função que não é primitiva recursiva. Ela pode ser definida no cálculo lambda utilizando o teorema da recursão, estruturas como o combinador Y (para recursão) e as funções básicas de números naturais e booleanos definidas em questões anteriores.

A definição da função de Ackermann pode ser estruturada como segue:

Definição Recursiva da Função de Ackermann

A função Ackermann é dada por:

$$\begin{aligned} Ar := \lambda r. \lambda m. \lambda n. & \text{ IF } (ISZERO\ m) (SUCC\ n) \\ & \left(\text{IF } (ISZERO\ n) (r\ r\ (PRED\ m) (SUCC\ ZERO)) \right. \\ & \left. (r\ r\ (PRED\ m) (r\ r\ m\ (PRED\ n))) \right) \end{aligned}$$

Definição da Função de Ackermann

Usando o combinador ω (um combinador equivalente ao Y), obtemos a função de Ackermann como segue:

$$Ackermann := \omega\ Ar$$

Descrição da Lógica

A lógica da função segue os casos clássicos:

- Caso base 1: Se $m = 0$, então $Ackermann(m, n) = n + 1$.
- Caso base 2: Se $n = 0$, então $Ackermann(m, n) = Ackermann(m-1, 1)$.
- Caso recursivo: Caso contrário, $Ackermann(m, n) = Ackermann(m-1, Ackermann(m, n-1))$.

Essa definição em lambda calculus é suficiente para implementar e calcular a função de Ackermann de forma puramente funcional.

6) 5 Exemplos de Funções Definidas na Forma Solicitada

Uma função $F : \mathbb{N}^n \rightarrow \mathbb{N}$ definida por recorrência no estilo das funções gerais recursivas de Kleene é um sistema de equações na forma abaixo, onde cada vetor $\vec{x}$ é a lista $x_1, \dots, x_n$, e cada t_{ij} é Suc , Z ou P_l^m , isto é, as funções sucessor, zero e projeção. Os termos E_i são formados por composição das funções básicas Suc , Z e P_{ij} , possivelmente incluindo a função F (daí a notação $E_i(F)\vec{x}$). Em geral, pode-se incluir uma função G já definida por recursão geral.

$$\begin{cases} F \circ \langle t_{11}, \dots, t_{n1} \rangle \vec{x} = E_1(F)\vec{x} \\ F \circ \langle t_{12}, \dots, t_{n2} \rangle \vec{x} = E_2(F)\vec{x} \\ \vdots \\ F \circ \langle t_{1k}, \dots, t_{nk} \rangle \vec{x} = E_k(F)\vec{x} \end{cases}$$

Pede-se:

1. Mostre pelo menos cinco exemplos já conhecidos de funções definidas desta forma, com suas respectivas definições;
2. Argumente que toda função primitiva recursiva pode ser definida de acordo com o esquema acima;
3. Argumente que toda função definida da forma acima é parcialmente recursiva.

Exemplos de Funções Definidas Desta Forma

A seguir, apresentamos cinco exemplos de funções conhecidas definidas segundo este esquema, com suas respectivas definições.

a) Função de Adição ($\text{Add}(x, y) = x + y$)

$$\begin{cases} \text{Add} \circ \langle x, Z \rangle = x \\ \text{Add} \circ \langle x, \text{Suc}(y) \rangle = \text{Suc}(\text{Add} \circ \langle x, y \rangle) \end{cases}$$

Em termos das funções básicas:

$$\begin{cases} \text{Add} \circ \langle P_1^2, Z \rangle = P_1^2 \\ \text{Add} \circ \langle P_1^2, \text{Suc}(P_2^2) \rangle = \text{Suc}(\text{Add} \circ \langle P_1^2, P_2^2 \rangle) \end{cases}$$

b) Função de Multiplicação ($\text{Mul}(x, y) = x \times y$)

$$\begin{cases} \text{Mul} \circ \langle x, Z \rangle = Z \\ \text{Mul} \circ \langle x, \text{Suc}(y) \rangle = \text{Add} \circ \langle x, \text{Mul} \circ \langle x, y \rangle \rangle \end{cases}$$

Onde Add é a função de adição definida anteriormente.

c) Função de Predecessor ($\text{Pred}(x) = x - 1$ com $\text{Pred}(0) = 0$)

$$\begin{cases} \text{Pred} \circ \langle Z \rangle = Z \\ \text{Pred} \circ \langle \text{Suc}(x) \rangle = x \end{cases}$$

Em termos das funções básicas:

$$\begin{cases} \text{Pred} \circ \langle Z \rangle = Z \\ \text{Pred} \circ \langle \text{Suc}(P_1^1) \rangle = P_1^1 \end{cases}$$

d) Função de Subtração Limitada ($\text{Sub}(x, y) = \max(0, x - y)$)

$$\begin{cases} \text{Sub} \circ \langle x, Z \rangle = x \\ \text{Sub} \circ \langle x, \text{Suc}(y) \rangle = \text{Pred} \circ (\text{Sub} \circ \langle x, y \rangle) \end{cases}$$

e) Função de Fibonacci ($\text{Fib}(n)$)

$$\begin{cases} \text{Fib} \circ \langle Z \rangle = Z \\ \text{Fib} \circ \langle \text{Suc}(Z) \rangle = \text{Suc}(Z) \\ \text{Fib} \circ \langle \text{Suc}(\text{Suc}(n)) \rangle = \text{Add} \circ \langle \text{Fib} \circ \langle \text{Suc}(n) \rangle, \text{Fib} \circ \langle n \rangle \rangle \end{cases}$$

2) Toda Função Primitiva Recursiva Pode Ser Definida Segundo Este Esquema

As funções primitivas recursivas são definidas a partir de funções básicas (zero, sucessor e projeções) através das operações de composição e recursão primitiva. O esquema apresentado permite:

- Definir funções básicas: Suc, Z, P_i^n .

- Realizar composição de funções.
- Definir funções por recursão, onde a função F aparece no lado direito da equação.

A recursão primitiva tem a forma:

$$\begin{cases} f(0, \vec{x}) = g(\vec{x}) \\ f(n+1, \vec{x}) = h(n, f(n, \vec{x}), \vec{x}) \end{cases}$$

Onde g e h são funções primitivas recursivas já definidas.

O esquema apresentado é capaz de expressar este tipo de definição, pois permite que $E_i(F)\vec{x}$ seja composto de F e outras funções básicas. Assim, qualquer função primitiva recursiva pode ser definida segundo este esquema.

3) Toda Função Definida Segundo Este Esquema É Parcialmente Recursiva

As funções parcialmente recursivas (ou funções recursivas gerais) incluem todas as funções computáveis, ou seja, aquelas que podem ser calculadas por uma máquina de Turing.

O esquema apresentado permite a definição de funções usando recursão geral, sem a restrição de que a recursão seja primitiva (que garante a terminação). Portanto, funções que podem não terminar (funções parcialmente computáveis) também podem ser definidas.

Além disso, o esquema permite a inclusão de funções G já definidas por recursão geral, aumentando o poder expressivo.

Assim, toda função definida segundo este esquema é, no mínimo, parcialmente recursiva, pois é computável por uma máquina de Turing.

Conclusão

O esquema de definição apresentado é suficientemente poderoso para representar várias funções conhecidas e importantes, como a adição, multiplicação, predecessor, subtração limitada e a sequência de Fibonacci. Ele também é capaz de expressar qualquer função primitiva recursiva, pois permite a composição e recursão necessárias para tal. Além disso, todas as funções definidas segundo este esquema são parcialmente recursivas, sendo computáveis por uma máquina de Turing.

7) Comentário sobre a Necessidade do Uso do Teorema da Recursão

Na resposta anterior, o **Teorema da Recursão** não foi explicitamente utilizado. Esse teorema, garante que, para qualquer função computável f , existe uma função g tal que $g = f(g)$.

No caso das funções apresentadas (adição, multiplicação, predecessor, subtração limitada e Fibonacci), foi utilizada **recursão primitiva**, que sempre termina e garante que as funções são totais e computáveis. A demonstração se baseia nas propriedades de fechamento das funções primitivas recursivas sob composição e recursão primitiva, sem a necessidade do teorema da recursão.

Para funções parcialmente recursivas, o teorema da recursão assegura que funções definidas por recursão geral (que podem não terminar) são computáveis. Assim, ele é fundamental no contexto de funções parciais, embora não tenha sido necessário na resposta, pois lidamos com recursão primitiva.

Conclusão: - Para **funções primitivas recursivas**, o uso do teorema da recursão não é necessário, pois elas são totais e baseiam-se em recursão primitiva. - Para **funções parcialmente recursivas**, o teorema da recursão fundamenta a definição de funções possivelmente não terminantes, mas não foi estritamente usado na resposta.

10) Teorema

Para todo λ -termo na forma M , existe uma configuração $T = (Q, \Gamma, b, \Sigma, \delta, q_0, F)$ de uma máquina de Turing, onde:

- Γ é um alfabeto finito de símbolos da fita;
- $b \in \Gamma$ é o símbolo vazio, sendo o único que pode aparecer repetidamente na fita de forma infinita;
- $\Sigma \subseteq \Gamma \setminus \{b\}$ representa o conjunto de símbolos de entrada, ou seja, aqueles que podem estar na fita no início da execução;
- Q é um conjunto finito de estados;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados de aceitação;
- δ é a função de transição da máquina.

Dado um λ -termo M , ao tentar reduzir M com um argumento x , a expressão não alcançará uma forma normal se, e somente se, a máquina de Turing representada por T não parar para a fita inicial contendo x . Por outro lado, se reduzirmos M com um argumento y , onde $y \neq x$, a expressão será avaliada a um resultado z se, e somente se, a máquina de Turing correspondente a T também parar com y como entrada e produzir z de forma equivalente na fita final.

11) Questões de Lambda-cálculo

Um λ -termo é normal se e somente se ele não possui nenhuma ocorrência de β -redex, isto é, um subtermo na forma $(\lambda x.M)N$. Marque (V) ou (F) justificando.

1. **(V)** O termo $(\lambda h.(\lambda x.h(xx))(\lambda x.h(xx)))N$, onde N é um λ -termo, **não** tem forma normal.

Justificativa: Esse termo é uma aplicação do combinador Y , que é usado para implementar recursão no λ -cálculo. Ao aplicar as reduções, o termo entra em um loop infinito sem alcançar uma forma normal. Portanto, ele não possui forma normal.

2. **(V)** Todos os tipos de dados apresentados em aula (Booleano, Lista, Números Naturais) são representados por termos em forma normal.

Justificativa: As representações dos dados usando as codificações de Church (como os numerais de Church para números naturais, os booleanos $\text{TRUE} = \lambda x.\lambda y.x$ e $\text{FALSE} = \lambda x.\lambda y.y$) são termos que não contêm β -redexes. Eles são expressos como abstrações cujos corpos não podem ser mais reduzidos. Portanto, esses termos estão em forma normal.

3. **(F)** Qualquer ordem de aplicações de β -redução a um λ -termo obtém sua forma normal.

Justificativa: Essa afirmação é falsa. Mesmo que o λ -cálculo seja confluente (pelo Teorema de Church-Rosser), nem toda estratégia de redução garante alcançar a forma normal se ela existir. Em particular, estratégias que aplicam reduções em subtermos que levam a computações infinitas podem impedir que a forma normal seja alcançada. Apenas estratégias de redução que evitam esses subtermos (como a redução em ordem normal) garantem chegar à forma normal se ela existir.

4. (V) Considere SUC como o λ -termo que implementa a função de sucessor sobre números naturais na representação de numerais de Church. Seja Y o termo $(\lambda h.(\lambda x.h(xx))(\lambda x.h(xx)))$. O termo $Y \text{ SUC } n$, onde n é um numeral de Church, **não** possui forma normal.

Justificativa: O termo Y é o combinador Y , usado para criar pontos fixos de funções no λ -cálculo. Quando aplicado a SUC e a um numeral n , o termo $Y \text{ SUC } n$ tenta encontrar um ponto fixo da função sucessor começando em n . Como a função sucessor não possui ponto fixo (não há número natural k tal que $k = k + 1$), a redução do termo entra em um loop infinito e nunca alcança uma forma normal.

12) Prova da Existência de um λ -termo K Tal que $K M \rightarrow K$

Enunciado

Prove que existe um λ -termo K , tal que, qualquer que seja o λ -termo M , $K M$ reduz-se a K . Argumente como este termo K pode ser visto como o programa que imprime a si mesmo.

Resposta

Definição do Termo K

Definimos o λ -termo K usando o combinador Y (combinador de ponto fixo) da seguinte forma:

$$K := Y (\lambda k. \lambda x. k)$$

Onde o combinador Y é definido por:

$$Y := \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

Demonstração de que $K M \rightarrow K$

Vamos mostrar que, para qualquer λ -termo M , temos:

$$K M \rightarrow K$$

Prova:

1. Começamos substituindo K por sua definição:

$$K = Y (\lambda k. \lambda x. k)$$

2. Aplicando o combinador Y :

$$K = (\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))) (\lambda k. \lambda x. k)$$

3. Simplificando a aplicação do combinador Y :

$$K = (\lambda x. (\lambda k. \lambda x. k) (x x)) (\lambda x. (\lambda k. \lambda x. k) (x x))$$

4. Observamos que, através das reduções, obtemos que K é um ponto fixo de $\lambda k. \lambda x. k$:

$$K = (\lambda k. \lambda x. k) K$$

$$K = \lambda x. K$$

5. Agora, aplicamos K a um λ -termo arbitrário M :

$$K M = (\lambda x. K) M$$

$$K M \rightarrow K$$

Portanto, para qualquer λ -termo M , temos que $K M$ reduz-se a K .

Interpretação de K como um Programa que Imprime a Si Mesmo

O termo K possui a propriedade de que, quando aplicado a qualquer termo M , o resultado da avaliação é novamente K . Em outras palavras, K é um ponto fixo que se reproduz independentemente da entrada.

No contexto da computação, podemos interpretar K como um programa que, não importando a entrada fornecida, retorna a si mesmo como saída. Isso é análogo a um programa que, quando executado, imprime seu próprio código fonte — um programa autorreplicante ou *quine*.

Conclusão

Demonstramos que existe um λ -termo K tal que, para qualquer λ -termo M , a aplicação $K M$ reduz-se a K . Além disso, argumentamos que K pode ser visto como um programa que imprime a si mesmo, p

13) Implementação de Arrays (Vetores) no Lambda Cálculo

É possível implementar arrays n -dimensionais utilizando a abordagem de listas encadeadas, conforme apresentado na questão 3. Para arrays de dimensões fixas, o marcador de lista vazia *NIL* pode ser descartado. Um array vazio de tamanho fixo n pode ser representado como uma sequência encadeada de $n - 1$ pares, onde cada par armazena um valor padrão, como *ZERO*. Isso ocorre porque o último nó da lista encadeada pode conter dois elementos em vez de um, eliminando a necessidade de um marcador para indicar o final da lista.

A indexação aleatória de elementos dentro do array pode ser realizada de forma simples, percorrendo a sequência encadeada e contando os nós até alcançar o índice desejado. Cada nó corresponde a uma posição do array, tornando o processo direto e eficiente para estruturas de tamanho fixo. Essa representação elimina a complexidade de gerenciar marcadores de fim de lista, aproveitando a simplicidade das listas encadeadas.

14) Implementação de Arrays Multidimensionais no Lambda Cálculo

Na questão anterior, discutimos a implementação de arrays unidimensionais (vetores) no λ -cálculo. Agora, vamos verificar se é possível implementar de forma *sucinta* arrays de mais de uma dimensão.

Definição de Sucinto

Entendemos por **sucinto** uma implementação que seja elegante, concisa e que não introduza complexidade desnecessária. Uma representação sucinta utiliza abstrações e construções simples para alcançar o objetivo, facilitando a compreensão e manipulação do conceito.

Implementação de Arrays Multidimensionais

É possível implementar arrays multidimensionais no λ -cálculo de forma sucinta ao generalizar a abordagem usada para arrays unidimensionais. Em vez de representar um array como uma função de um índice a um valor, podemos representar um array multidimensional como uma função que recebe múltiplos índices e retorna o valor correspondente.

Por exemplo, um array bidimensional (matriz) pode ser representado como:

$$A := \lambda i. \lambda j. \text{elements } i \ j$$

Onde *elements* é uma função que, dados os índices *i* e *j*, retorna o valor na posição (i, j) . Usando *currying*, tratamos *A* como uma função que recebe *i* e retorna outra função que recebe *j*.

A implementação de *elements* pode seguir a mesma ideia da versão unidimensional, utilizando condicionais aninhados para verificar os índices e retornar os valores correspondentes. Para cada dimensão adicional, adicionamos um parâmetro e aninhamos mais um nível de condicionais.

Considerações Finais

Embora seja possível implementar arrays multidimensionais desta forma, a representação permanece sucinta apenas para arrays de dimensões baixas e tamanhos pequenos. À medida que aumentamos o número de dimensões ou o tamanho dos arrays, a complexidade da implementação cresce significativamente, tornando-a menos elegante e prática. Portanto, a sucintez é mantida apenas em casos simples, e para arrays maiores pode ser necessário buscar abordagens mais sofisticadas.